

Mission Orientated Flight Software Development: CubeSTEP

Tirth Thakkar, Viren Kumar, Shridhi Seth, Navid Nakhjiri, Maranda Laws, Marco Maggia, Howard Lee.

California Polytechnic State University Pomona

Pomona, CA; +1 (858)-504-0209

tsthakkar@cpp.edu

ABSTRACT

Small satellite missions often distribute mission intent across concept of operations (CONOPS) documents, ground procedures, and flight software (FSW) mode logic, with limited formal coupling between them. In many small-team CubeSat missions, mission operations are finalized late and evolve independently of FSW, leading to misalignment between documented procedures, operator expectations, and autonomous behavior. This separation can introduce ambiguity during operations, complicate verification, and increase the risk that autonomous behaviors satisfy local safety constraints while diverging from mission-level objectives. This paper presents a mission-oriented approach in which mission operations artifacts, including concept of operations (CONOPS) and operations manual, are explicitly mapped to a custom-designed FSW architecture. The approach adopts a co-design workflow in which mission operations and FSW are developed together, ensuring alignment between operational intent, ground procedures, and onboard behavior.

The proposed approach organizes mission intent into discrete stages with defined objectives, entry and exit conditions, authority boundaries, and allowable actions. These stages are mapped to a constrained set of onboard finite state-machine states implemented within the FSW, including *Initialization*, *Safe*, *Idle*, *Experiment*, and *Transmit*. Each state is annotated with supported mission objectives, permitted resource usage, and prioritized fault responses. Transitions between states are governed by goal-aware guards that evaluate spacecraft health, attitude stability, power margins, communications status, and payload readiness. This structure establishes a verifiable contract between mission goals, operational procedures, and FSW behavior, enabling autonomy while preserving operator authority over irreversible or mission-ending actions.

The approach is demonstrated using a CubeSat mission concept (CubeSTEP-Cerberus) centered on an oscillating heat pipe thermal experiment, with the mission structured into five operational stages. The commissioning stage comprises initial deployment, detumble, antenna deployment, first beacon, and orbit determination, followed by a validation stage during which system checkout is performed, including power validation, RF verification, and thermal-sensing enablement. The experiments stage consists of experimental cycles using patch heaters and thermocouples with automated data capture, after which a reduced operation stage is entered upon completion of primary objectives. The mission concludes with decommissioning requiring explicit ground command. Within this structure, *Idle* serves as a nominal intermediate state when no experimental operations or anomalies are present, during which the FSW polls attached peripherals at a predefined interval while performing memory scrubbing. *Safe* is a goal-preserving containment state triggered by health degradation, while *Experiment* is reachable only when mission stage, power, thermal constraints, and communications policies are satisfied, with results queued for downlink in *Transmit*.

The FSW is implemented using the FPrime framework with FreeRTOS, developed directly from the mission operations manual, operational procedures, authority boundaries, and contingency responses traced to specific software states and transitions. Software-in-the-loop testing is used to validate autonomous behavior and operational workflows under representative fault scenarios. The result is a scalable method for CubeSat mission development in which mission operations and FSW are co-designed, mission intent is treated as a first-class artifact, and operational robustness and goal assurance are improved through architectural alignment.

INTRODUCTION

CubeSTEP-Cerberus is a student-led, multidisciplinary collaboration with the NASA Jet Propulsion Laboratory (JPL) within the Aerospace Engineering Department at California State Polytechnic University, Pomona. The project aims to design, build, test, and launch a modular 3U CubeSat, as shown in Figure 1, that will serve as an on-orbit testbed to advance the technology readiness level (TRL) of an oscillating heat pipe (OHP) thermal management system.

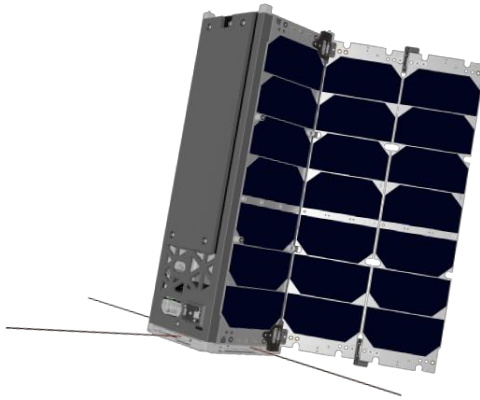


Figure 1: CubeSTEP-Cerberus 3U CubeSat

Problem Statement

Flight software is usually developed for extremely constrained, mission-specific contexts. That constraint is necessary, but it also makes the software brittle when mission operations change. In many CubeSat projects, this brittleness is not treated as a mission development problem early enough. A small change in an operating assumption, ground procedure, or subsystem interface can force the FSW team to rewrite local mode logic, command handlers, and fault responses instead of updating a defined mission model.

This creates a recurring development mismatch between mission scope and software capability. Operators think in terms of mission phases, science objectives, ground-pass opportunities, and data recovery. FSW is often implemented in terms of local modes, device commands, and threshold checks. When those two views are not coupled, different team members can build incompatible pieces of the same mission. For CubeSTEP-Cerberus, this gap is especially important because the primary objective depends on spending considerable system resources on repeatable thermal experiments while preserving the context of each data set across a planned two-year mission life.

The disconnect is exacerbated in student-led and small-team missions because subsystem development, operations planning, and software integration proceed on different schedules. Hardware interfaces, ground-station procedures, and payload constraints evolve during assembly and verification. In practice, important mission assumptions can become distributed across meeting notes, operating procedures, and separate code bases. The risk is that autonomous behavior satisfies a narrow subsystem rule while still violating the mission-level objective.

This paper therefore treats mission intent as a first-class software input. The CONOPS phases are translated into FSW states, and each state is assigned explicit entry conditions, exit conditions, resource limits, command authority, and fault responses. Instead of allowing the spacecraft to transition only on low-level events, the FSW evaluates goal-aware guards tied to spacecraft health, power margin, attitude stability, communication status, payload readiness, and data-storage state. The result is a traceable contract between mission operations and the onboard finite state machine. To establish this contract, the operational concepts traditionally captured in mission operations products must be represented in a form that the FSW can interpret and enforce.

Mission Operations Products

Mission operations products define how a spacecraft is commanded, monitored, and operated throughout its lifecycle. These products include the Concept of Operations (CONOPS), flight rules, command and telemetry dictionaries, commissioning procedures, experiment procedures, pass plans, anomaly response procedures, and decommissioning procedures. In the proposed approach, these products serve not only as a guideline for operators but also as structured aspects of the mission that can be translated into onboard software behavior.

MISSION CONTEXT AND CONSTRAINTS

Orbital and Communications Environment

Cerberus is planned as a low-Earth-orbit CubeSat mission that will be launched in a 510 km sun-synchronous orbit. These values define the expected operating environment, but from the software perspective the more important fact is that orbital state affects experiment validity. A thermal test performed during sunlight entry is not equivalent to one performed during mid-sunlight, sunlight exit, or umbra. Orbital phase therefore becomes a software guard, not just a mission description.

Communication is intermittent. Cerberus relies on a single UHF ground station at Cal Poly Pomona, giving approximately 2–3 expected contact opportunities per day at 9.6 kbps when visibility and link margin are sufficient. The spacecraft cannot assume operator availability when a fault occurs or when an experiment opportunity becomes

valid. Ground contact is therefore treated as a scheduled opportunity for command updates, telemetry review, and data recovery rather than a continuous control path.

Executing a ground pass begins with a pre-pass review. Once the duration of the pass and the availability of the ground station are confirmed, the team determines which commands are to be transmitted. This depends on mission objectives, planned activities, and contact time. When the satellite comes in view of the ground station, the first step is acquisition of signal. After the communication link is established, a health check is conducted to monitor the status of the spacecraft. If all reports are nominal, the approved command sequence is uplinked, and the prioritized data is downlinked. The data to be downlinked is determined ahead of time to ensure critical information is received even if the pass is interrupted. A post-pass assessment is performed after contact ends to evaluate results and prepare for future contact windows.

Resource Constraints

Power is the dominant resource constraint. Payload heating and radio transmission are high-energy activities. During eclipse, heater activity and unnecessary transmission must be inhibited to preserve battery state of charge. During sunlight, the spacecraft may charge and prepare for higher-power operations if the EPS reports adequate margin above the minimum operating threshold. Cerberus is expected to generate approximately 10 -30 W when properly exposed to sunlight.

Data storage is also a mission constraint. Each experiment must preserve science samples, configuration metadata, time tags, orbital-phase estimates, heater set point, OHP selection, and spacecraft health context. The spacecraft is limited to 2 MB of flash and 1 MB of RAM, while each contact window can recover only a limited amount of data. A valid experiment is therefore not only one where the heater turns on and thermocouples record data. It is one where the resulting data set can still be interpreted after downlink.

The nominal operations plan follows directly from these limits. The spacecraft charges to an acceptable margin, waits for the required lighting and resource conditions, executes a thermal test, stores the result with context, and downlinks the data during later ground contact. End-of-life operations are treated separately because decommissioning permanently changes the mission configuration. The FSW may support decommissioning by disabling payload activity, reducing power consumption, preserving beacon capability, and transitioning to final state, but final payload disablement and data closeout remain ground-authorized actions.

Hardware and Software Context

Cerberus uses an STM32-class onboard computer, FreeRTOS, and the F Prime flight software framework. The spacecraft interfaces include EPS telemetry, solar panel status, antenna and transceiver interfaces, ADC-based sensor acquisition, flash storage, patch heater control, thermocouple acquisition, and ADCS-related health information. These interfaces may mature at different times during integration, so mission behavior cannot be locked inside one monolithic control loop.

The software is therefore organized around a separation between hardware-facing components and mission-level decision logic. Hardware components report status and execute bounded actions. Mission-level components decide whether an action is currently allowed. This separation is what allows environmental conditions, subsystem limits, and operator authority to become flight rules instead of informal assumptions embedded across the code base.

MISSION CONCEPT OF OPERATIONS

The CONOPS is used as the first grouping of mission behavior. Each phase defines the current mission objective, the allowable spacecraft actions, the required authority level, and the conditions that must be satisfied before the spacecraft can move forward. In this paper, the CONOPS is not only a timeline. It is the first step in converting mission operations into software states and guarded transitions.

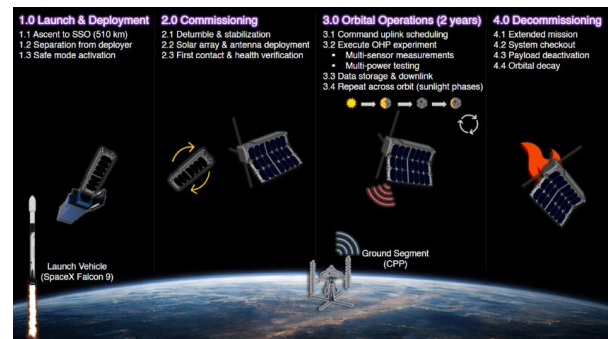


Figure 2: CubeSTEP-Cerberus Concept of Operations

Phase One: Launch, Deployment, and Stabilization

The mission begins when the CubeSat is ejected from its deployer. The deployment switch activates the power rails and the onboard computer begins initialization. The spacecraft is expected to be tumbling, the antenna is stowed, and no ground communication is available. The objective is survival-focused: initialize the spacecraft, reduce body rates, deploy required hardware, and establish first contact.

This phase is autonomous since no uplink path exists before antenna deployment and first beacon. The ADCS enters detumble behavior, the spacecraft waits for acceptable attitude stability, and the OBC commands antenna release and solar panel deployment. Once the antenna is available, Cerberus begins transmitting beacons with basic health and deployment status. Phase One ends when the first beacon is received and the initial orbit is confirmed.

Phase Two: Commissioning and Validation

After first contact, the spacecraft enters commissioning to determine if the system can safely support experimental operations. The EPS must show power-positive behavior, the UHF link must support reliable communication, data buses must return valid subsystem status, and thermal sensing paths must produce usable measurements.

The spacecraft may be physically capable of receiving payload commands before commissioning is complete, but those commands are not mission-valid until health, communication, and sensing paths have been verified. Exiting commissioning requires ground review and approval.

Phase Three: Orbital Operations and Experiments

The primary mission phase consists of a structured thermal experiment matrix. Cerberus evaluates three OHP variants, four orbital sunlight phases, and five heater power levels. The resulting set of experiments requires the spacecraft to activate the selected patch heater, record thermocouple data, capture cooldown behavior, store the result, and downlink the data for analysis.

An experiment is only valid when the selected OHP, heater setting, orbital phase, power margin, thermal state, storage status, and spacecraft health all satisfy the current mission rules. The experiment phase therefore creates the strongest need for guarded transitions and data-preservation behavior.

Phase Four: Reduced Operations and Decommissioning

Once the primary science objectives are complete, Cerberus enters reduced operations. New payload activity is disabled, housekeeping continues, and remaining data are recovered during contact windows. This phase protects spacecraft health while the team verifies received information.

Decommissioning is ground-authorized. Before a permanent payload-disable command is sent, operators must verify that required experiment data have been downlinked and archived. After payload disablement, the spacecraft transmits a final decommissioning beacon and enters a low-power configuration with only basic housekeeping and periodic beacon capability.

FLIGHT RULES AND DECISION FRAMEWORK

Definition of Flight Rules in Mission Context

Flight rules are the software-facing form of the mission constraints. A flight rule defines what the spacecraft is allowed to do under a given mission phase, health state, resource condition, and authority level. In implementation, a rule may appear as a command restriction, state-transition guard, threshold response, event flag, or ground-authorization requirement.

The need for flight rules follows from the context above. Intermittent communication requires autonomous health protection. Limited power requires inhibits on heater activity and transmission. Storage constraints require data-preservation behaviors. Commissioning requires science lockout until subsystems are verified. Decommissioning requires ground approval because the spacecraft should not autonomously decide that the mission is complete.

Table 1: Mission Constraints Converted into Software Rules

Mission constraint	Software rule or guard
Orbital phase affects experiment validity	Experiment entry requires the requested sunlight condition or approved override.
Intermittent communication	Spacecraft must maintain health, beacon status, and queue data without ground contact.
Limited power margin	Heater activity and high-rate downlink are inhibited below EPS thresholds.
Limited storage and downlink	Experiment data cannot overwrite undownlinked data without an approved rule.
Commissioning before science	Payload execution is locked out until required bus and sensor checks pass.
Decommissioning is irreversible	Final payload disablement and data closeout require ground authorization.

These rules prevent mission intent from being scattered across procedures and component code. Instead of each subsystem interpreting the mission independently, the FSW evaluates the same decision question at every transition: given the current phase, spacecraft health, available resources, and operator authority, what is the spacecraft allowed to do next?

Prioritization of Flight Rules

Flight rules are prioritized by their effect on spacecraft survival, mission continuity, data integrity, and payload performance. This ordering resolves conflicts: when objectives compete, the higher-priority rule governs the allowed response.

The highest priority is spacecraft survival. These rules protect battery state of charge, thermal limits, attitude stability, deployment safety, and basic communications. Survival rules override payload and downlink objectives and may be enforced autonomously because waiting for ground contact can introduce unacceptable risk.

The second priority is mission continuity. These rules preserve the spacecraft's ability to continue operating after off-nominal events. They include commissioning gates, attitude requirements before deployment-dependent actions, limits on repeated reset attempts, and health checks before high-impact commands.

The third priority is data integrity. For Cerberus, science value depends on both measurements and their operational context. Data-integrity rules preserve meta-data, prevent unsafe overwrites of undownlinked data, and record fault context when experiments are interrupted. A partial labeled data set is preferable to an unlabeled or overwritten one.

The lowest priority is payload performance. Payload rules determine when an experiment is valid or desirable, but payload activity is enabled only after survival, continuity, and data-integrity constraints are satisfied.

This hierarchy also defines command authority. Protective actions such as safe-mode entry, load shedding, experiment abort, beaconing, and preservation of partial data may occur autonomously. Mission-progress actions, including exiting commissioning, selecting experiment parameters, approving high-rate downlink, or returning from *Safe*, require operator approval. Irreversible actions, including deployment commands, permanent configuration changes, critical updates, payload disablement, and decommissioning, require explicit ground authorization.

This decision structure maps naturally to a finite state machine. Each state bounds a set of allowed functions, each transition represents a permitted path between functions, and each flight rule determines whether that path is currently valid. For example, low battery state of charge inhibits heater activity, restricts nonessential transmission, and holds or transitions the spacecraft to *Safe*. Excessive body rates or loss of stable attitude aborts payload activity, preserves partial experiment context, and requires attitude recovery before returning to *Idle*. Missed ground contacts limit unnecessary radio reset attempts, preserve stored data, and maintain periodic beaconing to support recovery without excessive power use.

MISSION-TO-SOFTWARE STRUCTURE

The prioritized flight rules are implemented as a finite state machine. CONOPS phases describe mission intent, while FSW states describe executable spacecraft behavior. These are related but not identical: one mission phase may use multiple software states, and one software state may support multiple mission phases.

Initialization supports launch, deployment, detumble, first beacon, and early subsystem setup. *Safe* contains degraded-health or invalid operating conditions. *Idle* is the nominal monitoring state, where the spacecraft polls subsystems, maintains health awareness, accepts allowed commands, and waits for experiment or communication opportunities. *Experiment* is entered only when mission phase, resource state, payload readiness, and data-storage guards are satisfied. *Transmit* is entered during valid communication windows to downlink housekeeping and stored mission data.

The state set is intentionally small. Rather than modeling every procedure as a top-level mode, the architecture defines a limited number of stable software states and places mission-specific detail inside state behavior or transition guards. This keeps future changes local: experiment updates remain in *Experiment*, communication updates remain in *Transmit*, and health-response updates remain in *Safe* transition logic.

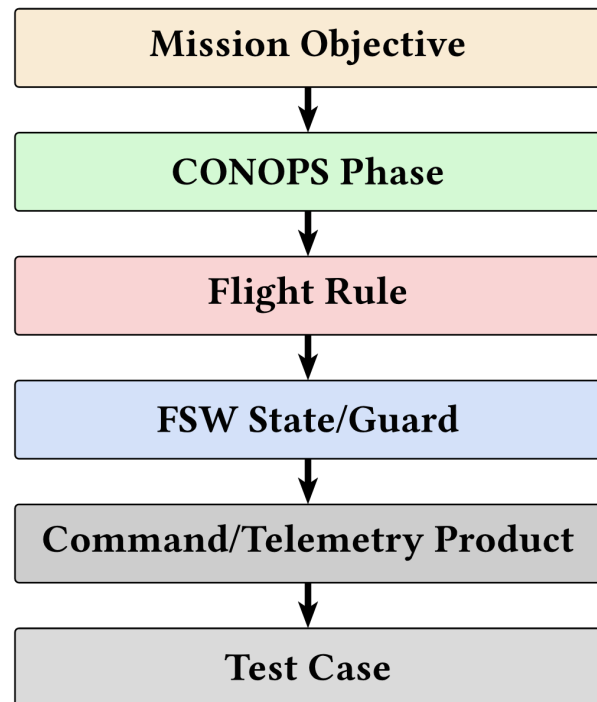


Figure 3: Traceability Diagram

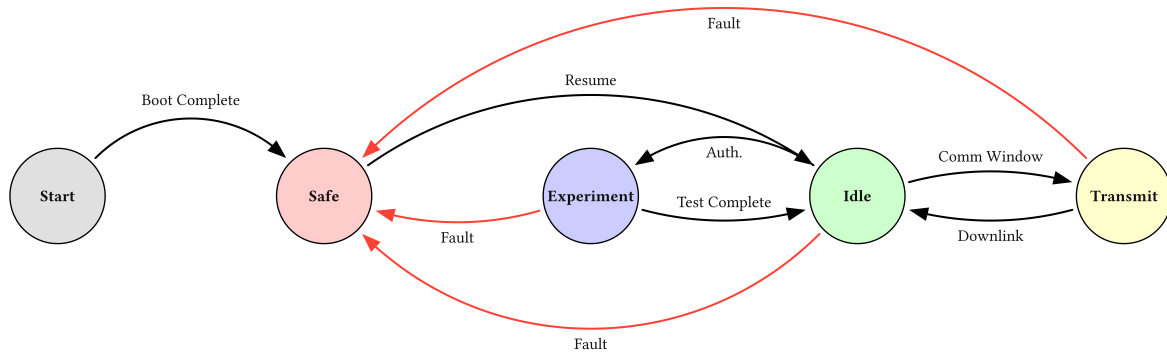


Figure 4: CubeSTEP-Cereberus Top-Level Flight Software State Machine

FLIGHT SOFTWARE AND BEHAVIORAL LOGIC

Initialization State

The initialization state is entered immediately after deployment. The spacecraft first executes detumbling until body rates fall within allowable limits. Once stable, the FSW commands solar panel deployment, releases the antenna, and begins beaconing to establish first ground contact. During this state, the FSW initializes mission time, thermal variables, subsystem status flags, and scheduled health checks. After required subsystem readiness checks are complete, initialization is logged and the spacecraft transitions to *Safe* for post-boot health verification.

Safe State

Safe is a protective state that may be entered from any other state when spacecraft health violates defined limits. In *Safe*, payload activity is inhibited and the spacecraft operates with minimum required functionality while preserving power, communication capability, and recoverability. Once health, power, attitude, and communication conditions return within acceptable limits, the spacecraft transitions to *Idle*. Ground operators may queue recovery or experiment-related commands while the spacecraft remains in *Safe*, but execution is deferred until the relevant state and flight rules allow it.

Idle State

Idle is the nominal waiting state and the entry point for all routine operations. In *Idle*, the FSW performs scheduled health checks, peripheral polling, memory scrubbing, and ground-contact monitoring. *Experiment* commands may be queued by operators, but execution is delayed until the spacecraft satisfies the guards for *Experiment*. If a valid ground-station contact is detected, the spacecraft transitions to *Transmit*.

Experiment State

The experiment state contains setup, execution, and closeout substates. During setup, the FSW verifies operator inputs of OHP variant, sunlight phase, and heater

power level and then checks temperature, power margin, attitude stability, and storage availability meet required baseline conditions. Once the entry guards are satisfied, the spacecraft transitions to execution, where thermocouple data is collected. At completion, the FSW logs the result, resets elapsed experiment time, stores the data package, and returns to *Idle*. If safety limits such as battery or temperature are violated the experiment becomes invalid, the spacecraft aborts the sequence and transitions to *Safe*.

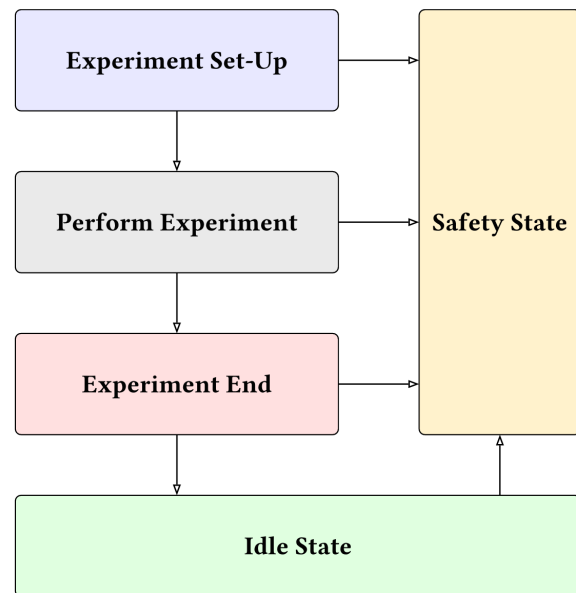


Figure 5: Experimental State Diagram

Transmit State

The transmit state is entered during valid ground-station contact to downlink mission data and receive approved commands. If contact is lost, the spacecraft returns to *Idle*. If detumbling or another safety-critical condition occurs, transmission is terminated and the spacecraft transitions to *Safe*.

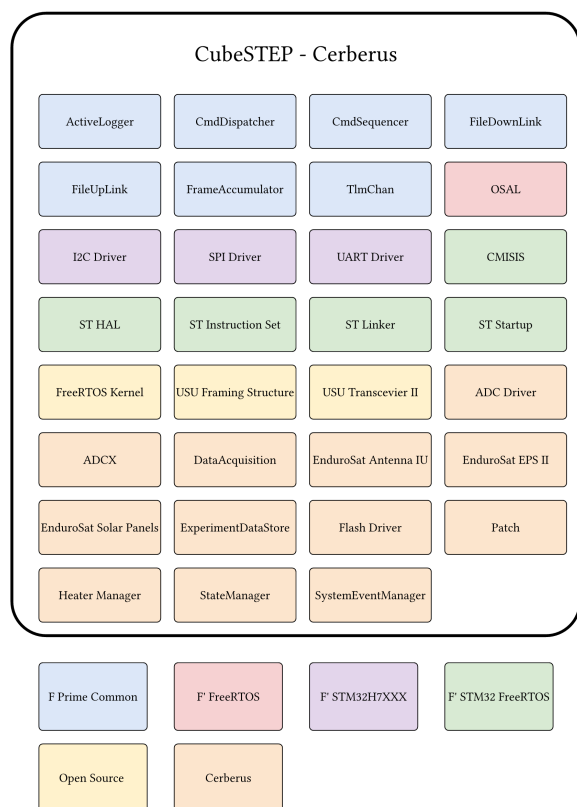


Figure 6: Cerberus Software Architecture Map

SOFTWARE IMPLEMENTATION

Tools and Methodology

F⁺ was chosen as the core flight software framework for Cerberus given the open source and readily available nature of public references, on-going support and maintenance by both JPL and the community, alongside the support of the NASA M-STAR grant funding for the mission. With the core real-time operating system of FreeRTOS being chosen due to the noted compatibility with the system OBC.

A major part of the software effort was adapting the F Prime FreeRTOS path for the Cerberus use case. Two reusable implementation elements were developed as part of this effort. The first is the shared F Prime/FreeRTOS support layer, centered on the operating system abstraction layer and runtime configuration needed to execute F Prime components under FreeRTOS. The second is the STM32 deployment layer, which connects the FreeRTOS runtime to STM32 startup code, linker configuration, CMSIS, ST HAL, and low-level peripheral interfaces. Together, these elements allow mission components to remain largely independent of the underlying board configuration while still supporting deployment to embedded hardware.

F Prime Common Services

F Prime common components are used for command, telemetry, logging, sequencing, framing, and file transfer. CmdDispatcher receives and dispatches commands from the ground system to the appropriate flight software components. CmdSequencer supports stored or preplanned command execution when operational procedures require a sequence of actions rather than a single command. TlmChan publishes telemetry channels used by the ground station to monitor spacecraft health, mission state, experiment progress, and subsystem status. ActiveLogger records events generated by the state machine, subsystem components, fault monitors, and command validation logic.

The file and communication path is implemented through FileUplink, FileDownlink, and FrameAccumulator. FileUplink supports the transfer of command products or configuration updates to the spacecraft, while FileDownlink supports recovery of stored experiment data and housekeeping records. FrameAccumulator collects framed data products received through the radio interface and passes them into the F Prime command and data handling path.

F Prime FreeRTOS and STM32 Platform Support

The F Prime FreeRTOS support layer provides the operating system adaptation needed for embedded execution. The OSAL maps F Prime runtime concepts onto FreeRTOS services, including task execution, timing, synchronization, and memory behavior. This layer was improved and adapted for the Cerberus deployment so that F Prime components can be supported on any STM32H7 platform.

The STM32 platform layer provides the hardware-specific foundation required by the deployment. This includes CMSIS, ST HAL, startup files, linker configuration, and instruction-set support. These elements define how the software initializes the processor, configures memory, accesses peripherals, and enters the FreeRTOS scheduler. The I2C, SPI, and UART drivers provide the primary communication interfaces to spacecraft subsystems.

The FreeRTOS kernel provides the runtime scheduler for concurrent spacecraft tasks. Periodic health monitoring, command processing, telemetry publication, experiment sampling, file operations, and subsystem polling are organized as scheduled activities. Priority-based scheduling is used for time-sensitive functions, while lower-priority activities such as routine telemetry packaging and background memory operations can execute when higher-priority spacecraft functions are not active. This supports the mission-rule hierarchy: survival and health-monitoring behavior must preempt payload and downlink activities when spacecraft conditions degrade.

Cerberus Mission Components

The Cerberus-specific layer contains the components that implement spacecraft behavior, subsystem interfaces, payload operation, and mission-rule enforcement. These components are the point where the CONOPS-derived flight rules become executable software.

StateManager implements the top-level finite state machine used by the mission. It manages transitions among *Initialization*, *Safe*, *Idle*, *Experiment*, and *Transmit*, and it enforces the allowable paths between those states. SystemEventManager monitors health and event inputs from subsystem components and raises state-transition requests or command inhibits when mission rules are violated. For example, if power, thermal, attitude, communication, or payload-readiness conditions fall outside allowable limits, SystemEventManager can trigger a transition to *Safe* or prevent entry into *Experiment*.

The payload execution path is implemented through HeaterManager, DataAcquisition, ExperimentDataStore, ADC interfaces, thermocouple-related acquisition logic, and the flash storage path. Patch and Patch Heater control the heater-side behavior required for OHP experiment execution. DataAcquisition collects experiment and housekeeping measurements during payload operation. ExperimentDataStore packages experiment results with required metadata, including experiment configuration, time context, heater setting, spacecraft health, and storage status. The Flash Driver provides the persistent storage interface used to retain experiment data until it can be downlinked.

Subsystem-facing components provide software interfaces to the EnduroSat EPS II, EnduroSat solar panels, EnduroSat antenna interface unit, transceiver, ADC hardware, and ADCX. These components expose health and status information to the state machine and event manager. They also allow the software to enforce flight rules at the subsystem boundary. For example, payload activation can be inhibited when EPS telemetry indicates insufficient power margin, downlink can be delayed when antenna or transceiver status is not valid, and deployment or recovery actions can be restricted until attitude and subsystem health conditions are acceptable.

As an additional layer of mission resilience, the flight software architecture supports a controlled in-flight patching capability. Patches are limited, ground-authorized modifications intended to correct verified software issues, update configuration parameters, or adjust mission logic within predefined safety boundaries.

All patching actions are constrained by the same authority model used for flight rules and state transitions. A patch may be sent only after properly mapping to mission

intent documentation and validation against the current spacecraft state. Patches that affect survival behavior, deployment logic, payload disablement, or other irreversible functions require explicit review and adjustment to mission intent. All requests are rejected if spacecraft is outside *Idle*.

Testing and Verification

Verification enforces mission intent under both nominal and off-nominal conditions. The first level of testing consists of unit testing of individual F Prime components and their respective software features using the GTest framework. These tests simulate and validate that command structure, if-checks, threshold monitors, file-writing operations, and mission behavior functions as intended.

The second level is the validation of the flight software on the same hardware architecture as the OBC through the creation of a software deployment on a commercially available development board sharing a processor of the same family as the system used on the spacecraft. Additionally, a debug deployment is created for the actual flight hardware as will be used on the mission.

The third level is component-level integration testing in which utilizes the aforementioned deployment platforms alongside a standard Linux environment. This tiered approach allows for the rapid evaluation and testing of core software elements within local development environments, before transitioning to a representative hardware platform. Once testing on these platforms yields satisfactory results, confirming that system resource limitations and differing tool chain, have no noted impact on telemetry, packetization and transmission, or no asserts are triggered in the actual mission application context. This integration method is utilized to validate all hardware components, on CubeSTEP-Cerberus those being EnduroSat Solar Panels 3U, EPS, EnduroSat Antenna I, Tensor Tech ADCSX MTQ, EnduroSat Transceiver II, and EnduroSat EPS II.

These comprehensive verification methods ensure that core baseline functional requirements are met for all hardware devices and their core software interfaces. Ultimately, unit tests ensure that software states remain strictly aligned with recorded CONOPS behavior patterns, significantly increasing the likelihood of overall mission success. To manage unforeseen anomalies, during the mission, the system supports in-flight code patching. As supporting such a functionality is a high-risk capability, it is strictly controlled and requires explicit ground-station authorization. Multiple operational safeguards are implemented to validate and verify all patches before execution, ensuring they do not compromise system stability.

DISCUSSION

The current MDO/FSW development effort is best characterized as an architectural integration activity between mission planning and embedded implementation. The MDO side defines mission phases, experiment opportunities, ground-pass usage, and data recovery priorities. The FSW side converts those decisions into executable states, command validation rules, telemetry products, and test scenarios. The remaining work before launch is to replace assumed thresholds with verified subsystem values from hardware testing, finalize pass-planning tools, confirm payload data rates and storage margins, and validate state transitions against the integrated spacecraft.

The approach is scalable because it does not depend on the exact CubeSTEP orbit or a single payload sequence. If launch conditions, pass opportunities, or thermal constraints change, the mission rules can be updated as configuration and verification artifacts rather than being scattered throughout procedural text and source code. The main limitation is that mission-rule quality depends on the maturity of subsystem constraints. Early in the project, the architecture can enforce only conservative placeholders; as test data improve, those placeholders must be updated to avoid either unsafe permissiveness or overly restrictive operations.

CONCLUSION

This paper presents a mission-oriented FSW development method for CubeSTEP-Cerberus. The central problem is the separation between CONOPS, ground procedures, and autonomous onboard behavior that can occur in small satellite projects. The proposed solution maps mission phases to a constrained FSW state machine and attaches objective, authority, resource, telemetry, and fault-response information to each state. For an OHP technology-demonstration mission, this mapping is critical because experiment validity depends on controlled heater inputs, thermal measurements, orbital-phase context, and reliable data recovery.

By developing MDO and FSW together, the project converts mission intent into executable software constraints. The state machine provides a compact onboard behavioral model, while mission phases preserve operator-level context across the two-year campaign and end-of-life sequence. The result is a verifiable path from mission objective to software state, command guard, telemetry product, and SITL test case. For a student-led CubeSat project, the principal educational value is not only the resulting software architecture, but also the discipline of treating operations, autonomy, and verification as one coupled design problem.

Acknowledgments

The authors gratefully acknowledge Celeste Smith and the F Prime team at the NASA Jet Propulsion Laboratory for their invaluable support throughout this work. The authors also extend sincere thanks to, Jeffrey W. Levison, Kevin Ortega, Dr. Takuro Daimaru, Dr. Scott Roberts, and Dr. Jeremiah Gayle for their technical guidance, encouragement, and continued support. This work was supported by the NASA Minority University Research and Education Project (MUREP) under Grant No. 80NSSC23M0223.

REFERENCES

- [Amazon Web Services, 2026] Amazon Web Services (2026). FreeRTOS Kernel. <https://github.com/FreeRTOS/FreeRTOS-Kernel>. Accessed: May 29, 2026.
- [Khandekar et al., 2003] Khandekar, S., Dollinger, N., and Groll, M. (2003). Understanding operational regimes of closed loop pulsating heat pipes: An experimental study. *Applied Thermal Engineering*, 23(6):707–719.
- [Larson and Steel, 2026] Larson, B. and Steel, G. (2026). Radio Communication Subtopology for F Prime.
- [Levison et al., 2018] Levison, J. W., Reder, L. J., Watney, G. J., Canham, T. K., and Bocchino, Robert L., J. (2018). F Prime: An open-source framework for small-scale flight software systems. Technical Report NASA NTRS Document ID 20210008573, Jet Propulsion Laboratory, National Aeronautics and Space Administration, Pasadena, CA, USA.
- [National Aeronautics and Space Administration, 2016] National Aeronautics and Space Administration (2016). *NASA Systems Engineering Handbook*. NASA.
- [The CubeSat Program, Cal Poly SLO, 2014] The CubeSat Program, Cal Poly SLO (2014). *CubeSat Design Specification Rev. 13*. California Polytechnic State University, San Luis Obispo.